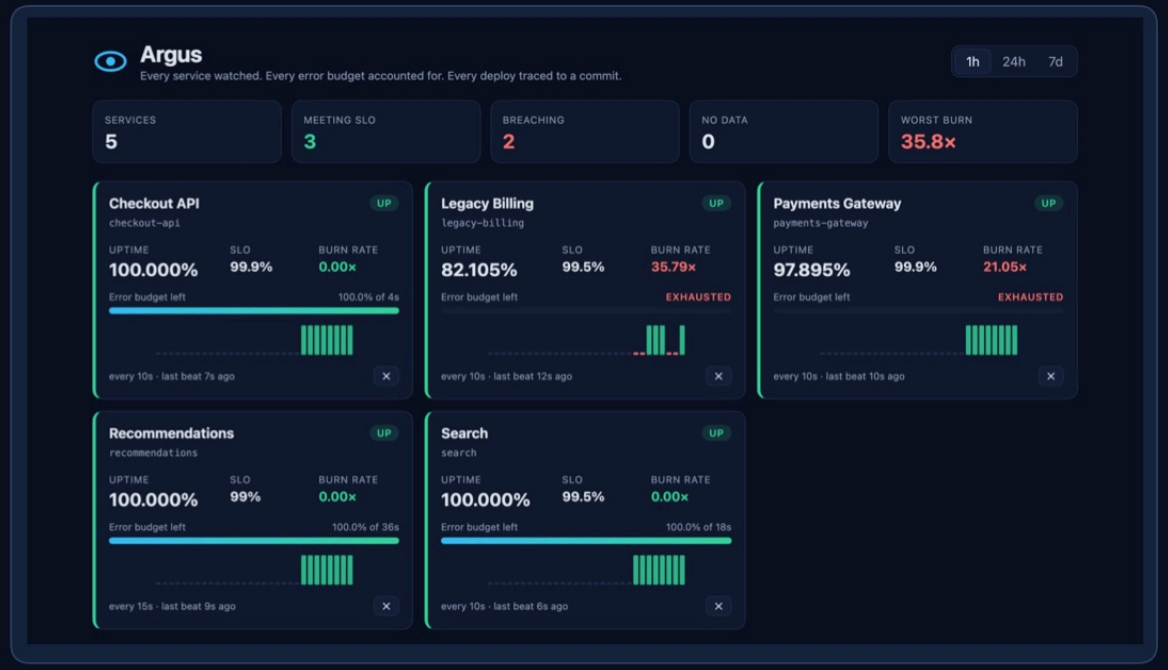
Argus

SLO and uptime monitoring, shipped through a complete DevOps pipeline

Every service watched. Every error budget accounted for. Every deploy traced to a commit.

Arnav Yadav
Roll no. 24BCS10424
Capstone project
github.com/arnav-yadav/argus



The live status board, served through the Kubernetes Ingress

Liveness is inferred from silence

1. **Register**

   Name, heartbeat interval and SLO target in; a one-time token out, stored only as a hash

2. **Beat**

   Every interval: ok, degraded or fail, with the token. A wrong token gets a 401

3. **Silence = down**

   Miss two intervals and the service is DOWN, though it never said so

4. **Compute SLOs**

   Uptime, error budget and burn rate over 1h, 24h and 7d

86.4 s — daily error budget at a 99.9% SLO: (1 − 0.999) × 86,400 s

1× — burn rate that spends exactly the budget over the window

14.4× — burn rate that pages someone: a 30-day budget gone in about two days

Built with FastAPI · PostgreSQL + Alembic · React + nginx · 48 tests, 98% coverage

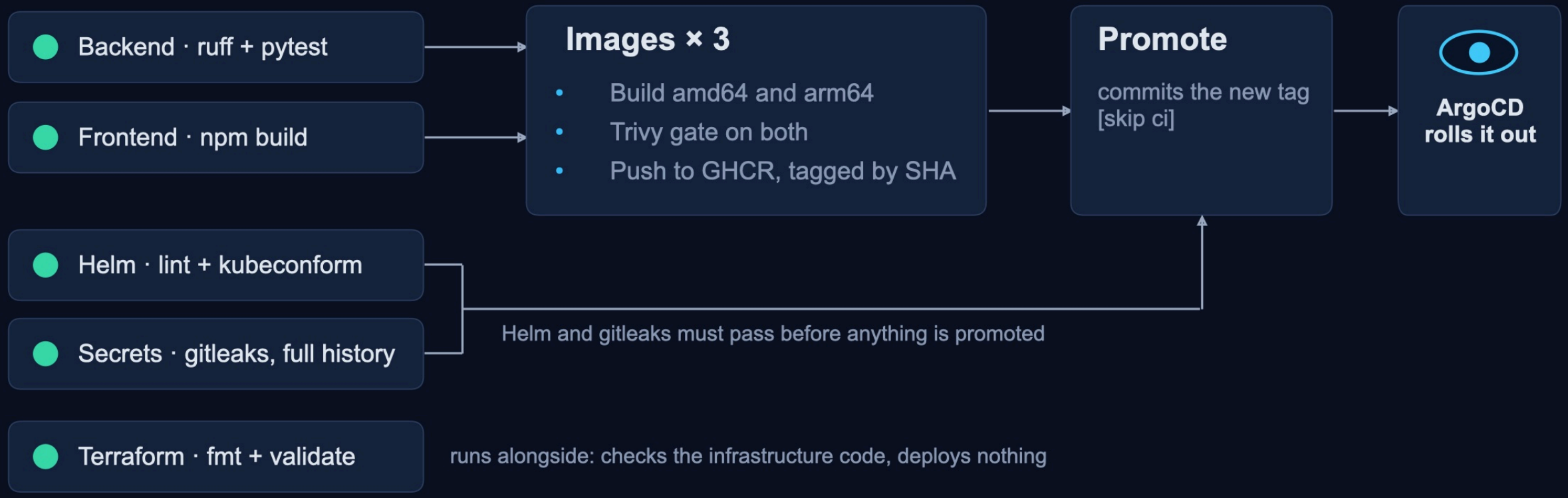
THE DELIVERY CHAIN

From commit to cluster in 13 stages

1. **Application** — FastAPI + React
2. **Git commit** — one change each
3. **CI pipeline** — GitHub Actions
4. **Docker image** — multi-stage, non-root
5. **Security scan** — Trivy, amd64 + arm64
6. **Registry** — GHCR, SHA tags
7. **Terraform** — AWS VPC + EKS
8. **Kubernetes** — minikube and EKS
9. **Helm** — one chart, 2 environments
10. **Ingress** — / and /api
11. **Autoscaling** — HPA, 2 to 6
12. **Monitoring** — Prometheus, Grafana
13. **Troubleshooting** — real incidents, fixed

The stages the course asks the final demo to cover. Every one is built, and evidenced in docs/EVIDENCE.md.

Every push to main ships in under 4 minutes



Trivy blocked run 1: 4 HIGH CVEs in pip's bundled libraries, so pip was removed

Alpine backend: 306 → 178 MB, 0 vulnerabilities, non-root

Tags are commit SHAs, never latest; GitHub holds no cluster credentials

From Terraform to autoscaling

07 · TERRAFORM

AWS, built and torn down

52

resources: a VPC and an EKS 1.35 cluster

- 2 public + 2 private subnets, one NAT gateway
- Encrypted node disks, API allow-listed to one IP
- Audit logs, node auto repair
- Applied, evidenced, destroyed: under $1

08–10 · K8S, HELM, INGRESS

One chart, two environments

2

environments: main by ArgoCD, staging by helm upgrade

- Ingress: / to the frontend, /api to the backend
- Postgres as a StatefulSet with a PVC
- Pod Security restricted: non-root, read-only filesystem

11 · AUTOSCALING

The HPA under real load

2 → 6

backend replicas in 30 seconds

- 70% CPU target; the load peaked at 383%
- 12 clients for 3 minutes, through the Ingress
- 0 heartbeats missed while scaling

Monitored, alerted, debugged



Grafana dashboard, provisioned from a ConfigMap

Fast burn
14.4× over 1h + 5m

Medium burn
6× over 6h + 30m

Slow burn
1× over 24h + 1h

Three real incidents, three root causes

Every service merged into one Prometheus series
Cause our service label clashed with Kubernetes' own
Fix honorLabels on the ServiceMonitor

Every deploy looked like an outage
Cause two simulators rotated each other's tokens mid-rollout
Fix Recreate strategy for the simulator

terraform apply stopped on one resource
Cause the AWS org policy denies OIDC providers
Fix IRSA off, EKS Pod Identity instead

LIVE DEMO

A commit deploys itself

1. **Commit** — change the tagline and push
2. **Pipeline** — tests, images, Trivy, GHCR: about 3 min
3. **Promote** — a bot commits the new tag
4. **ArgoCD** — notices within a minute, then syncs
5. **Live** — pods roll; the board shows the new SHA

48	0	52	38
tests, 98% coverage	vulnerabilities in 6 image builds	AWS resources, destroyed after	screenshots in docs/EVIDENCE.md



Thank you · Questions?

github.com/arnav-yadav/argus